MDI3003

ADVANCED PREDICTIVE ANALYTICS

STUDENT LABORATORY INSTRUCTION MANUAL

Experiment 07

Constructing a Recommendation System from Customer Transaction Data using Random Forest

From transaction history to candidate generation, purchase-propensity scoring, Top-K ranking, and responsible recommendation

Manual element	Specification
Scheduled duration	3 hours
Primary method	Random Forest classifier for user-item purchase-propensity scoring and ranking
Core baselines	Popularity / most-purchased items; optional co-purchase heuristic
Core evaluation	Precision@K, Recall@K, Hit Rate@K, MAP@K/NDCG@K (extension), ROC-AUC/PR-AUC for pair classification
Advanced learner methods	Collaborative Filtering, Matrix Factorization, LightFM, Neural Collaborative Filtering, Two-Tower / retrieval models
Recommended environment	Python 3.10+;

Faculty: Dr. Durgesh Kumar

Assistant Professor (Senior), SCOPE, VIT Vellore

Fall Semester 2026-2027

Revision 2.0 - Benchmark-incorporated edition

This revision strengthens the three-hour core with a frozen dataset extract, fixed chronological cutoffs, a bounded candidate catalog, explicit candidate-recall reporting, and clearer advanced benchmarking against canonical recommender methods.

Technical positioning: Random Forest is not a native collaborative-filtering algorithm. In this laboratory it is used as a supervised candidate-scoring model: construct user-item examples from transaction history, estimate purchase/relevance probability, rank candidate items for each customer, and return Top-K recommendations. This distinction is essential for technical correctness.

How to Use This Manual

- Complete the pre-lab reading and verify that the instructor-provided dataset extract loads before the scheduled laboratory.
- Use the 3-hour core to build a reproducible baseline-to-Random-Forest recommendation pipeline on one transaction dataset.
- Do not use future transactions when constructing features for earlier recommendation decisions.
- Evaluate recommendations with ranking metrics, not only classification accuracy.
- Complete collaborative filtering, matrix factorization, neural recommenders, cross-dataset replication, and fairness/diversity analysis as advanced extensions. For any advanced comparison, use the same frozen split and candidate policy, and report uncertainty across at least three random seeds or a bootstrap 95% confidence interval.

Core principle: A recommendation system is evaluated by whether relevant future items appear near the top of the ranked list. A high pair-classification accuracy can be meaningless when negatives dominate.

Contents

1. Laboratory Positioning

This laboratory treats recommendation as a ranking problem over customer-item candidates. Historical transaction data are converted into time-valid features. A Random Forest classifier estimates the probability that a customer will purchase or interact with a candidate item in a future evaluation window; candidate items are then sorted by score to produce personalized Top-K recommendations.

Why this framing matters: Random Forest can learn nonlinear interactions among recency, frequency, monetary value, customer preferences, item popularity, category affinity, and co-purchase signals. It does not by itself discover latent user-item factors, so collaborative filtering and matrix factorization are retained as advanced comparisons.

2. Aim and Learning Outcomes

2.1 Aim

To construct, evaluate, and interpret a personalized recommendation system from customer transaction data using Random Forest as the primary supervised ranking/scoring model.

2.2 Learning outcomes

- Transform transaction logs into leakage-safe user-item learning examples.
- Build popularity and simple heuristic recommendation baselines.
- Engineer customer, item, interaction, recency, frequency, monetary, and affinity features.
- Train and tune a Random Forest classifier using only past information.
- Generate personalized Top-K ranked recommendations.
- Evaluate both pair-classification quality and recommendation-ranking quality.
- Analyze cold-start, popularity bias, sparsity, temporal drift, and failure cases.
- Compare Random Forest with collaborative filtering or matrix-factorization methods in the advanced extension.
- Document data provenance, privacy risks, reproducibility artifacts, and responsible-use boundaries.

3. Problem Statement and Business Context

An e-commerce or retail business wants to recommend products to customers based on previous transaction history. For each customer at a chosen recommendation time, the system must generate a small candidate set of items, compute features using only information available before that time, predict purchase/relevance likelihood, rank the candidates, and return the Top-K items.

The central research question is: Can a supervised Random Forest ranking pipeline outperform a non-personalized popularity baseline on future-item recommendation while remaining reproducible, leakage-safe, and computationally practical?

3.1 Predictive translation

Recommendation decision: Rank eligible products for a customer from past transactions.

Target: Future purchase/interaction in a clearly defined evaluation window.

Positive pair: Customer-item pair purchased/interacted with in the future window.

Negative pair: Eligible candidate not purchased in the future window; sampled carefully.

Success criterion: Relevant future items appear in Top-K more often than under the popularity baseline.

4. Three-Hour Scope and Instructor Checkpoints

Time	Core activity	Required evidence
0-20 min	Load instructor-frozen transaction extract; verify exact date range/cutoffs; audit customers, items, cancellations/returns, and missing IDs	Dataset card + checksum/version + fixed-cutoff confirmation
20-40 min	Apply fixed chronological train/validation/test windows; construct customer and item histories without future information	Time-window diagram + no-future-feature check
40-60 min	Build popularity baseline and bounded candidate-generation rule (about 500-2,000 eligible items)	Baseline Top-K list + candidate-policy record + candidate-recall check
60-95 min	Create user/item/interaction/RFM-style features and positive/negative user-item examples	Feature dictionary + class balance
95-125 min	Train Random Forest; training-only tuning / validation	Validation table + selected model
125-150 min	Generate Top-K recommendations on locked test customers	Recommendation table
150-170 min	Evaluate Precision@K, Recall@K, HitRate@K; inspect feature importance and errors	Metrics + plots + five cases
170-180 min	Cold-start/privacy discussion, save/reload artifacts, instructor check-off	Final checklist + reproducibility manifest

4.0 Core execution constraints

To keep the three-hour laboratory reproducible and computationally feasible, the instructor must provide a frozen transaction extract, fixed chronological cutoff dates, and an eligible candidate catalog capped to approximately 500-2,000 products. All groups use the same cutoffs and candidate policy.

4.1 Instructor check-offs

Check-off	Evidence
1 - Data and temporal validity	Verified dataset source; timestamp field; cancellation/return rule; chronological split; no future leakage.
2 - Baseline and candidate generation	Popularity baseline implemented; candidate universe defined; already-purchased-item policy documented.
3 - Random Forest scoring and ranking	Feature matrix, negative sampling, model validation, Top-K generation, ranking metrics.
4 - Interpretation and responsible use	Five recommendation cases, cold-start handling, popularity-bias note, privacy boundary, reload test.

5. Verified Public Datasets

The following datasets were verified as publicly accessible at the time this manual was prepared. The instructor should still provide a frozen extract or exact version/hash so that every student evaluates the same data.

Dataset	Verified public description	Recommended use
D1 - UCI Online Retail	541,909 transactions from a UK non-store online retailer, covering 01-Dec-2010 to 09-Dec-2011. Contains invoice, product code/description, quantity, invoice date, unit price, customer ID and country.	Preferred core dataset. Build customer-product future-purchase examples from invoice history.
D2 - Retailrocket Recommender System Dataset	Real e-commerce implicit-feedback data with 2,756,101 events from 1,407,580 visitors: views, add-to-cart and transactions; also item properties and category tree.	Advanced recommendation benchmark with richer implicit feedback and candidate signals.
D3 - Instacart Market Basket Analysis	Public grocery order-history benchmark widely used for next-basket prediction; contains orders, products, aisles/departments and prior order-product interactions.	Advanced next-basket / repeat-purchase recommendation and temporal feature engineering.
D4 - UCI Online Retail II (optional)	Two years of real online retail transactions; approximately 1.07 million instances.	Cross-dataset replication or robustness extension.

5.1 Dataset selection rule

- Core lab: use the instructor-frozen UCI Online Retail extract or an equivalently structured transactional dataset. The instructor should publish the exact file/hash, date range, train/validation/test cutoff dates, and eligible-product filtering rule before the session.
- Advanced learners: replicate the protocol on Retailrocket or Instacart. Do not claim direct metric comparability unless target definitions and candidate-generation policies are harmonized.
- Record dataset URL, access date, license/terms, version or file hash, filters, date range, number of customers, number of items, and number of interactions.

6. Data Schema, Privacy, and Transaction Integrity

Field type	Examples	Use
Customer identifier	CustomerID / visitorid / user_id	Grouping only; may be encoded for joins, not treated as an ordinal numeric feature.
Item identifier	StockCode / itemid / product_id	Candidate identity; joins and history construction.
Transaction/order identifier	InvoiceNo / transactionid / order_id	Basket grouping and duplicate control.
Timestamp	InvoiceDate / timestamp / order_number/time	Chronological split and recency features.
Quantity/value	Quantity, UnitPrice, transaction amount	Frequency/monetary features; handle returns/cancellations.
Item metadata	Description, category, aisle, department	Content/category affinity features when available.

6.1 Integrity rules

- Remove or explicitly model cancellations/returns; never silently count a returned item as a positive recommendation outcome.
- Remove rows without a usable customer identifier for personalized modelling, while documenting the number removed.
- Deduplicate exact duplicate lines when justified; preserve legitimate repeated quantities within an order.
- Use timestamps to ensure features are computed strictly before the prediction/evaluation window.
- Do not expose names, addresses, emails, payment information, or direct identifiers if present.

6.2 Dataset card

Item	Student entry
Source URL and access date	
Version/hash	
Raw rows	
Filtered rows	
Unique customers	
Unique items	
Date range	
Return/cancellation policy	
Missing-ID policy	
Privacy/usage note	

7. Recommender-System Foundations

7.1 Recommendation as ranking

For each customer u, a recommender scores candidate items i and returns the K items with the highest scores. In implicit-feedback transaction data, a future purchase can be treated as relevance evidence rather than as an explicit rating.

7.2 Core families

Family	Core idea	Role in this lab
Popularity / rank-based	Recommend globally or segment-wise popular items.	Mandatory simple baseline.
Content-based	Recommend items similar to a user profile built from item attributes.	Optional when item metadata exist.
Collaborative filtering	Use user-item interaction patterns and similarities.	Advanced comparison.
Matrix factorization	Learn latent user and item vectors.	Advanced benchmark.
Supervised candidate scoring	Predict relevance/purchase for user-item pairs using engineered features.	Primary Random Forest formulation.
Neural retrieval/ranking	Learn embeddings or deep interaction functions.	Advanced learner extension.

7.3 Why Random Forest can be useful

- Captures nonlinear interactions among customer, item and context features.
- Handles mixed numerical features and threshold effects with limited scaling requirements.
- Provides feature importance and tree-level interpretability aids.
- Can incorporate transaction-derived signals that pure collaborative filtering may not directly use.
- However, it needs an explicit candidate-generation and negative-sampling design and is not naturally optimized for global ranking.

8. Random Forest Formulation for Recommendation

Construct a supervised dataset of customer-item pairs. For a cutoff time t, compute features from history H<t and define y=1 when customer u purchases item i in the future target window; otherwise y=0 for sampled eligible candidates. The Random Forest estimates P(y=1 | x_{u,i,t}). Candidates are sorted by this score.

8.1 Example features

Level	Features
Customer	purchase count, unique items, recency, frequency, monetary value, average basket size, active days, preferred categories
Item	global purchase count, unique buyers, recent popularity, price statistics, category, repeat-purchase rate
Customer-item	previous purchase count, days since last purchase, share of customer spend, repeat indicator, co-purchase frequency
Context	day/month/season, country/region, basket context when available

8.2 Random Forest essentials

A Random Forest fits many decision trees to bootstrap samples and random subsets of features, then aggregates their predictions. For a binary purchase target, the predicted probability from the forest can be used as a ranking score.

```
score(u, i) = RandomForest.predict_proba(features(u, i))[:, 1]
recommend(u, K) = top_K candidate items ranked by score(u, i)
```

Important: Do not train on every possible customer-item pair. The negative class can become enormous. Use a documented candidate-generation rule and reproducible negative sampling, and evaluate ranking over a clearly defined candidate universe.

9. End-to-End Workflow

1. Define the recommendation moment and future target window.
2. Load and audit transactions.
3. Clean cancellations/returns and unusable customer IDs.
4. Create chronological train/validation/test windows.
5. Construct historical customer and item statistics from training history only.
6. Build a simple popularity baseline.
7. Generate candidate items for each customer.
8. Create positive and sampled-negative user-item pairs.
9. Build leakage-safe features.
10. Train/tune Random Forest using training/validation data.
11. Score candidates for locked-test customers.
12. Filter ineligible/already-purchased items according to policy.
13. Rank by predicted purchase probability and return Top-K.
14. Evaluate ranking metrics against future purchases.
15. Inspect feature importance, cold-start and failure cases.
16. Save model, feature schema, candidate policy, predictions and metrics.

10. Detailed Laboratory Procedure

10.1 Environment and reproducibility

```
# Core packages
import numpy as np, pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.model_selection import ParameterGrid
import joblib, json, platform, sklearn

SEED = 42
np.random.seed(SEED)
```

10.2 Load and normalize the core dataset

```
PATH = 'INSTRUCTOR_PROVIDED/online_retail.csv'
df = pd.read_csv(PATH, encoding='ISO-8859-1')
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
required = {'InvoiceNo','StockCode','Quantity','InvoiceDate','UnitPrice','CustomerID'}
missing = required - set(df.columns)
assert not missing, f'Missing columns: {missing}'
```

10.3 Basic transaction cleaning

```
# Example UCI Online Retail policy; document any changes.
df = df.dropna(subset=['CustomerID']).copy()
df['CustomerID'] = df['CustomerID'].astype(str)
df['is_cancel'] = df['InvoiceNo'].astype(str).str.startswith('C')
df = df[(~df['is_cancel']) & (df['Quantity'] > 0) & (df['UnitPrice'] >= 0)].copy()
df['Amount'] = df['Quantity'] * df['UnitPrice']
```

11. Candidate Generation and Negative Sampling

11.1 Chronological windows

Choose fixed dates or quantiles to create history/training, validation-target and locked-test-target windows. All features for a decision must use only transactions before that decision time.

```
t1 = df['InvoiceDate'].quantile(0.70)
t2 = df['InvoiceDate'].quantile(0.85)
train_hist = df[df.InvoiceDate < t1]
val_future = df[(df.InvoiceDate >= t1) & (df.InvoiceDate < t2)]
test_future = df[df.InvoiceDate >= t2]
```

Preferred classroom practice: The instructor should freeze exact cutoff dates rather than allowing each student to choose different quantiles.

11.2 Candidate universe

- Start with items sufficiently frequent in the training history (e.g., top N items or items with at least M buyers).
- Optionally exclude items that are no longer available or have no recent interactions.
- Decide whether repeat-purchase items are allowed; document this policy.
- For each customer, include all true future positives in the evaluation candidate set whenever feasible. If the candidate generator excludes some future positives, quantify and report candidate recall so ranking metrics are not interpreted without retrieval-stage context.

11.3 Candidate-recall audit

Before judging the Random Forest ranker, measure whether the candidate generator can retrieve the items that actually become relevant in the future window. Candidate Recall is the fraction of true future-positive items that are present in the evaluation candidate universe. A low value places an upper bound on achievable Recall@K and must be reported separately from ranking quality.

Candidate Recall = |Future relevant items ∩ Candidate set| / |Future relevant items|

Core requirement: report aggregate candidate recall and the number/percentage of evaluation users for whom all future positives are representable. Do not silently drop unretrievable positives.

11.4 Negative sampling

```
def sample_negatives(user, positives, candidate_items, n_neg=50, rng=None):
  rng = np.random.default_rng(SEED) if rng is None else rng
  pool = np.array(list(set(candidate_items) - set(positives)))
  n = min(n_neg, len(pool))
  return rng.choice(pool, size=n, replace=False).tolist()
```

12. Feature Engineering

12.1 Customer features

```
def customer_features(hist, cutoff):
  g = hist.groupby('CustomerID')
  out = g.agg(
    cust_txns=('InvoiceNo','nunique'),
    cust_items=('StockCode','nunique'),
    cust_qty=('Quantity','sum'),
    cust_spend=('Amount','sum'),
    cust_last=('InvoiceDate','max')
  ).reset_index()
  out['cust_recency_days'] = (cutoff - out['cust_last']).dt.days
  return out.drop(columns='cust_last')
```

12.2 Item features

```
def item_features(hist, cutoff):
  g = hist.groupby('StockCode')
  out = g.agg(
    item_txns=('InvoiceNo','nunique'),
    item_buyers=('CustomerID','nunique'),
    item_qty=('Quantity','sum'),
    item_avg_price=('UnitPrice','mean'),
    item_last=('InvoiceDate','max')
  ).reset_index()
  out['item_recency_days'] = (cutoff - out['item_last']).dt.days
  return out.drop(columns='item_last')
```

12.3 Customer-item features

```
def pair_features(hist):
  return (hist.groupby(['CustomerID','StockCode'])
      .agg(pair_purchases=('InvoiceNo','nunique'),
        pair_qty=('Quantity','sum'),
        pair_spend=('Amount','sum'),
        pair_last=('InvoiceDate','max'))
      .reset_index())
```

Leakage rule: Every aggregate must be recomputed from the history available at that fold/cutoff. Never compute global item popularity or customer statistics using validation/test transactions.

13. Model Training and Selection

13.1 Popularity baseline

```
popular_items = (train_hist.groupby('StockCode')['InvoiceNo']
          .nunique().sort_values(ascending=False).index.tolist())

def popularity_recommend(seen_items, k=10):
  return [i for i in popular_items if i not in seen_items][:k]
```

13.2 Random Forest model

```
rf = RandomForestClassifier(
  n_estimators=300,
  max_depth=None,
  min_samples_leaf=2,
  max_features='sqrt',
  class_weight='balanced_subsample',
  random_state=SEED,
  n_jobs=-1
)
rf.fit(X_train, y_train)
val_score = rf.predict_proba(X_val)[:,1]
```

13.3 Training-only tuning

Parameter	Core values to compare
n_estimators	200, 400
max_depth	None, 12, 20
min_samples_leaf	1, 2, 5
max_features	sqrt, 0.5
class_weight	balanced_subsample or justified alternative

Selection rule: Select hyperparameters using validation ranking performance (preferably Recall@K or NDCG@K) plus PR-AUC as a secondary pair-classification diagnostic. Do not choose the model from locked-test results.

14. Top-K Recommendation Generation

```
def recommend_for_user(user_id, candidate_df, model, feature_cols, k=10):
  u = candidate_df[candidate_df.CustomerID == str(user_id)].copy()
  if u.empty:
    return u
  u['score'] = model.predict_proba(u[feature_cols])[:,1]
  return u.sort_values('score', ascending=False).head(k)
```

15. Evaluation Metrics

Pair-classification metrics diagnose the scorer, but recommendation quality must be reported using ranking metrics over future relevant items.

Metric	Definition / interpretation	Core?
Precision@K	Relevant recommended items / K. Measures purity of the Top-K list.	Yes
Recall@K	Relevant recommended items / all relevant future items for the user.	Yes
Hit Rate@K	Fraction of users with at least one relevant item in Top-K.	Yes
MAP@K	Mean average precision across users; rewards relevant items appearing earlier.	Advanced
NDCG@K	Position-discounted ranking quality normalized by the ideal ranking; primary advanced ranking metric.	Advanced - primary
Coverage	Fraction of catalog/users meaningfully served.	Extension
ROC-AUC	Pair-level discrimination; can look strong under heavy class imbalance.	Diagnostic
PR-AUC	Pair-level precision-recall trade-off; more informative under imbalance.	Diagnostic
Candidate Recall	Fraction of future-relevant items that survive candidate generation. Diagnoses retrieval-stage loss before ranking.	Yes

```
def precision_at_k(recs, relevant, k):
  recs = recs[:k]
  return len(set(recs) & set(relevant)) / max(k, 1)

def recall_at_k(recs, relevant, k):
  if not relevant: return np.nan
  return len(set(recs[:k]) & set(relevant)) / len(set(relevant))

def hit_rate_at_k(recs, relevant, k):
  return float(len(set(recs[:k]) & set(relevant)) > 0)
```

16. Error, Cold-Start, and Leakage Analysis

Failure mode	Diagnostic question	Mitigation / discussion
Popularity domination	Does RF recommend the same few products to almost everyone?	Compare catalog coverage, personalization, item popularity distribution.
Cold-start customer	No usable transaction history?	Fall back to popularity, segment, country, or contextual recommendations.
Cold-start item	New item has no interaction history?	Use content/category metadata or business-approved exploration.
Repeat-purchase bias	Are consumables recommended repeatedly while discovery disappears?	Separate repeat vs novel recommendation metrics.
Temporal leakage	Were future popularity or RFM features included?	Recompute features at each historical cutoff.
Negative sampling bias	Are sampled negatives too easy/unrealistic?	Use candidate-aware negatives and report sampling policy.
Return/cancellation contamination	Are returned items labelled as positive purchases?	Apply explicit return policy before target construction.
Sparse users	Metrics unstable for one-purchase customers?	Minimum-history rule or separate cohort analysis.

16.1 Required five-case audit

- A successful personalized recommendation with supporting history.
- A relevant item missed by RF but found by popularity baseline.
- A case where RF beats popularity.
- A sparse/cold-start customer.
- A questionable recommendation caused by popularity, negative sampling, or feature limitations.

17. Visualization Requirements

- 1. Transaction volume over time.
- 2. Top 15 items by transaction count.
- 3. Customer purchase-frequency distribution.
- 4. Customer recency/frequency/monetary distributions.
- 5. Class balance after negative sampling.
- 6. Feature-importance chart for the selected Random Forest.
- 7. Precision@K and Recall@K versus K (e.g., K=5,10,20).
- 8. Popularity baseline versus Random Forest ranking metrics.
- 9. Recommendation-score distribution for positives and negatives.
- 10. Catalog-coverage or recommendation-popularity plot (extension).

Figure standard: Every figure must have a descriptive title, labelled axes, readable scale, and two or three sentences interpreting what the figure means for recommendation quality or risk.

18. Core and Advanced Experiments

Experiment	Requirement
E1 - Baseline	Popularity Top-K on locked evaluation customers.
E2 - RF recommendation	Random Forest user-item scorer with time-valid features.
E3 - K sensitivity	Compare K=5, 10, 20 using Precision@K/Recall@K/HitRate@K.
E4 - Feature ablation	Remove pair-history or customer RFM features and compare.
E5 - Negative sampling sensitivity	Compare at least two negative-sampling ratios or policies.
E6 - Advanced CF	Item-item collaborative filtering or implicit matrix factorization.
E7 - Advanced latent model	SVD/ALS/LightFM benchmark.
E8 - Neural extension	Neural Collaborative Filtering or Two-Tower retrieval/ranking model.
E9 - Cross-dataset	Replicate on Retailrocket/Instacart with harmonized target and candidate policy.
E10 - Responsible recommendation	Popularity bias, coverage, novelty/diversity or subgroup performance audit.
E6 - Candidate recall audit	Report candidate recall before ranker metrics; quantify future positives excluded by candidate generation.

18.1 Advanced Learner Pathway

Leading recommender-systems coursework treats collaborative filtering and latent-factor models as canonical recommender families. Therefore, advanced learners must compare Random Forest with at least one recommendation-native benchmark: item-item collaborative filtering or matrix factorization/ALS. LightFM, Neural Collaborative Filtering, and Two-Tower retrieval are additional optional extensions.

Method	Why compare it	Suggested metric
Item-item collaborative filtering	Strong interpretable interaction-based baseline.	Recall@K, NDCG@K
Matrix Factorization / ALS	Learns latent customer-item factors; strong implicit-feedback baseline.	Recall@K, MAP@K/NDCG@K
LightFM	Hybrid latent model using interactions plus metadata.	Recall@K, coverage
Neural Collaborative Filtering	Learns nonlinear user-item interaction functions.	NDCG@K, runtime
Two-Tower model	Separates user and item encoders for scalable retrieval.	Recall@K, latency, candidate retrieval speed

Advanced comparison question: Does the Random Forest gain over popularity justify its feature-engineering and scoring cost? Does a latent or neural recommender improve ranking enough to justify additional complexity and infrastructure?

Advanced statistical requirement: repeat stochastic models for at least three random seeds or report a user-level bootstrap 95% confidence interval for the principal ranking metric. Use Recall@K and NDCG@K as the principal advanced comparison metrics; classification ROC-AUC is diagnostic only.

19. Result Templates

Model	Precision@5	Recall@5	HitRate@5	Precision@10	Recall@10	HitRate@10	PR-AUC
Popularity							
Random Forest							
Advanced model							

Feature set	Validation Recall@10	Test Recall@10	Test NDCG@10	Observation
All core features				
Without pair history				
Without customer RFM				
Without item popularity				

Customer	History size	True future items	Top-5 recommendations	Hits	Comment
Case 1					
Case 2					
Case 3					
Case 4					
Case 5					

System	Training time	Scoring latency/user	Model size	Catalog coverage	Complexity note
Popularity					
Random Forest					
CF/MF/Neural extension					

19.5 Candidate-generation and uncertainty report

Report candidate recall, number of eligible products, percentage of users with all future positives representable, and the fixed candidate-policy version. Advanced comparisons must also report mean ± standard deviation across seeds or a bootstrap 95% confidence interval for Recall@K/NDCG@K.

20. Discussion Questions

17. Why is recommendation a ranking problem rather than only a binary-classification problem?
18. Why can classification accuracy be misleading in user-item data?
19. How does candidate generation change the recommendation problem?
20. What information leakage can occur when computing item popularity?
21. Why are random train/test splits risky for transaction recommendation?
22. How does negative sampling affect Random Forest probabilities and ranking?
23. Why might popularity outperform a personalized model for sparse users?
24. What is the difference between Precision@K and Recall@K?
25. Why should K be chosen based on product context?
26. How does the already-purchased-item policy affect repeat-purchase domains?
27. What does feature importance reveal—and what can it not prove?
28. How can popularity bias reduce catalog discovery?
29. When is collaborative filtering preferable to a feature-based Random Forest?
30. Why are matrix factorization models natural for sparse user-item interactions?
31. How should a recommender handle new customers or items?
32. What privacy risks arise from customer transaction profiling?
33. Why can a ranker with high Recall@K still fail if candidate recall is low?
34. Why should NDCG@K and uncertainty intervals be preferred over a single ROC-AUC when comparing advanced recommenders?
35. When might recommendation personalization become manipulative or discriminatory?
36. How should performance be monitored when product inventory and customer preferences drift?

21. Viva Questions and Expected Key Points

What is a recommendation system? A system that ranks/selects items likely to be relevant to a user under a defined context.

Why is Random Forest not a classical collaborative-filtering method? It requires engineered user-item features and supervised labels rather than directly factorizing or comparing the interaction matrix.

What is implicit feedback? Observed behavior such as purchase, view or add-to-cart rather than an explicit rating.

What is candidate generation? Creating the subset of eligible items that will be scored/ranked.

What is negative sampling? Selecting non-positive user-item pairs to train/evaluate a binary relevance model.

Why chronological splitting? Recommendations must predict future behavior using past information only.

Define Precision@K. Fraction of Top-K recommended items that are relevant.

Define Recall@K. Fraction of relevant future items recovered in Top-K.

What is HitRate@K? Whether a user has at least one relevant item in Top-K, averaged across users.

Why PR-AUC? More informative than ROC-AUC when positives are rare.

What is cold start? Insufficient interaction history for a new user or item.

What is popularity bias? Tendency to over-recommend already-popular items.

What is coverage? How much of the user/catalog space receives recommendations.

Why exclude future popularity? It leaks test-period information.

What are RFM features? Recency, frequency and monetary measures of customer behavior.

Why can feature importance be misleading? It indicates model dependence, not causality; correlated features can distort attribution.

What is collaborative filtering? Recommendation from user-item interaction similarity/patterns.

What is matrix factorization? Learning low-dimensional user/item latent vectors whose interaction predicts preference.

What is NDCG@K? Position-discounted ranking quality normalized by ideal ranking.

What is MAP@K? Mean average precision across users up to rank K.

Why filter unavailable items? Recommendations must be actionable and eligible.

Why save the candidate policy? Metrics depend strongly on the candidate universe.

What is a two-tower model? Separate user and item encoders producing embeddings for scalable retrieval.

What is temporal drift? Changes in preferences, catalog, pricing or behavior over time.

What is the human/business role? Set eligibility/policy constraints, monitor risk, and avoid harmful automated decisions.

22. Common Mistakes

Mistake	Why it is wrong	Correct practice
Random customer-level split with future transactions mixed into training	Temporal leakage and unrealistic evaluation.	Use chronological cutoffs and time-valid features.
Using classification accuracy as the main recommender metric	Negatives dominate; ranking quality is what users experience.	Report Precision@K/Recall@K/HitRate@K; NDCG/MAP extension.
Computing popularity/RFM on the entire dataset	Leaks validation/test information.	Recompute from each training history window.
Dropping all non-purchased pairs as negatives	Creates unrealistic/easy negatives and huge class imbalance.	Define candidate universe and reproducible sampling.
Recommending already-purchased items without policy	May be wrong for non-repeatable products.	Document repeat-vs-novel recommendation rule.
Random Forest scores every catalog item naively	Expensive and may create poor candidates.	Use candidate generation first.
Ignoring returns/cancellations	False positive relevance signals.	Apply explicit transaction-cleaning policy.
Comparing models with different candidate sets	Metrics are not comparable.	Use the same candidate policy and locked split.
Treating feature importance as causal	Predictive association is not causal effect.	Use it only for model interpretation.
Using customer IDs as continuous numeric values	Injects arbitrary ordinal structure.	IDs are keys; use history-derived features or embeddings.

23. Responsible Recommendation and Academic Integrity

- Data minimization: use only transaction fields needed for the learning objective.
- Do not expose identifiable purchase histories in screenshots or shared reports.
- Avoid recommendations that reveal sensitive inferred interests to unintended viewers.
- Assess popularity bias, concentration, and unfair exclusion of less-active customers or niche items.
- Do not use the system for discriminatory pricing, protected-class targeting, credit decisions, or manipulative personalization.
- Recommendation is a prediction of likely relevance, not proof of preference or intent.
- Document any external code, AI assistance, tutorials, datasets, or pretrained models used.

24. Submission Guidelines

- RegistrationNumber_Lab07_Recommender_RF.ipynb
- RegistrationNumber_Lab07_Report.pdf
- RegistrationNumber_Lab07_Ranking_Metrics.csv

RegistrationNumber_Lab07_Candidate_Recall.csv

- RegistrationNumber_Lab07_Recommendations.csv
- RegistrationNumber_Lab07_Error_Analysis.csv

RegistrationNumber_Lab07_Advanced_Uncertainty.csv (advanced learners)

- RegistrationNumber_Lab07_README.md
- models/random_forest.joblib
- artifacts/feature_schema.json
- artifacts/split_manifest.json
- artifacts/candidate_policy.json
- figures/*.png

25. Assessment Rubric - 100 Marks

Criterion	Marks	Observable evidence
Problem formulation & recommendation boundary	6	Correct ranking formulation, target window, candidate and repeat-purchase policy.
Dataset provenance & transaction integrity	7	Verified source/version, cleaning, returns/cancellations, missing IDs, privacy.
Temporal split & leakage prevention	10	Instructor-fixed chronological split, exact cutoffs recorded, time-valid aggregates, frozen test window, no future leakage.
Popularity baseline	6	Correct Top-K baseline and ranking metrics.
Candidate generation & negative sampling	10	Bounded eligible catalog (about 500-2,000 for core), reproducible sampling, future-positive inclusion policy, candidate recall reported.
Feature engineering	12	Customer, item and pair/RFM features with clear dictionary and no leakage.
Random Forest implementation & validation	14	Sound parameters/tuning, class imbalance handling, saved/reloadable model.
Top-K ranking & recommendation logic	10	Correct scoring, eligibility filtering and personalized Top-K output.
Evaluation & visualizations	10	Precision/Recall/HitRate@K, pair diagnostics, appropriate plots and interpretation.
Error/cold-start/bias analysis	7	Five-case audit, sparse-user handling, popularity bias and failure reasoning.
Responsible recommendation	4	Privacy, misuse, fairness/concentration and limitations.
Reproducibility & artifacts	3	Seed, manifest, schema, candidate policy, environment and reload checks.
Report quality	1	Concise, evidence-based and technically correct.

Total = 100 marks.

26. Final Submission Checklist

- ☐ Dataset source/version/hash and date range recorded.
- ☐ Cancellations/returns and missing IDs handled explicitly.
- ☐ Chronological train/validation/test windows saved.
- ☐ No future transaction information appears in features.
- ☐ Popularity baseline reported.
- ☐ Candidate-generation and negative-sampling policies saved.

☐ Core candidate catalog is bounded to the instructor-approved eligible set (approximately 500-2,000 products).

☐ Candidate recall is reported and unretrievable future positives are not silently discarded.

- ☐ Random Forest selected using validation data, not locked test.
- ☐ Top-K recommendations generated for eligible test customers.
- ☐ Precision@K, Recall@K and HitRate@K reported.
- ☐ At least five recommendation cases manually inspected.
- ☐ Cold-start and popularity-bias limitations discussed.
- ☐ Model saved and successfully reloaded.
- ☐ Notebook runs from top to bottom in a clean runtime.
- ☐ External sources/AI/code assistance disclosed.

☐ Advanced comparison includes item-item collaborative filtering or matrix factorization/ALS and reports Recall@K/NDCG@K with ≥3 seeds or bootstrap 95% CI.

27. References and Student-Help Links

Resource	Link	Use
UCI Online Retail	https://archive.ics.uci.edu/dataset/352/online+retail	Verified transaction dataset for the core lab.
UCI Online Retail II	https://archive.ics.uci.edu/dataset/502/online+retail+ii	Larger two-year transaction dataset for replication.
Retailrocket Recommender System Dataset	https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset	Implicit-feedback views/cart/transaction benchmark.
Instacart Market Basket Analysis	https://www.kaggle.com/competitions/instacart-market-basket-analysis/data	Public order-history benchmark for next-basket prediction.
scikit-learn RandomForestClassifier	https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html	Random Forest implementation and parameters.
scikit-learn Precision-Recall metrics	https://scikit-learn.org/stable/modules/model_evaluation.html	Pair-classification diagnostic metrics.
Stanford CS246 - Mining Massive Data Sets	https://web.stanford.edu/class/cs246/	Recommendation systems, collaborative filtering and latent factor methods.
Stanford CS224W recommender slides	https://web.stanford.edu/class/cs224w/slides/12-recsys.pdf	Top-K recommendation framing and Recall@K.
CMU 10-601 Introduction to Machine Learning	https://www.cs.cmu.edu/~mgormley/courses/10601/	Includes recommender systems and ensemble-method coverage.
MIT recommendation-system program overview	https://professional-education-gl.mit.edu/mit-online-data-science-program	Covers evaluation, sparsity, content/collaborative filtering, matrix estimation and neural approaches.

Appendix A - Core Notebook Scaffold

The following scaffold illustrates the required architecture. Students must adapt dataset paths and complete the pair-construction/evaluation functions for the instructor-frozen schema.

```
# 1. Load and clean
df = pd.read_csv(PATH, encoding='ISO-8859-1')
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
df = df.dropna(subset=['CustomerID'])
df = df[(~df.InvoiceNo.astype(str).str.startswith('C')) & (df.Quantity > 0)]
df['Amount'] = df.Quantity * df.UnitPrice

# 2. Freeze temporal cutoffs
# Instructor should provide exact dates.
train_end = pd.Timestamp('YYYY-MM-DD')
val_end   = pd.Timestamp('YYYY-MM-DD')
train_hist = df[df.InvoiceDate < train_end]
val_future = df[(df.InvoiceDate >= train_end) & (df.InvoiceDate < val_end)]
test_future = df[df.InvoiceDate >= val_end]

# 3. Build popularity baseline from train_hist only
popular = (train_hist.groupby('StockCode').InvoiceNo.nunique()
        .sort_values(ascending=False).index.tolist())

# 4. Build customer, item and pair features from history only
# Join to positive + sampled negative candidate pairs.
# X_train, y_train, X_val, y_val = ...

# 5. Fit Random Forest
rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=2,
                    max_features='sqrt', class_weight='balanced_subsample',
                    random_state=SEED, n_jobs=-1)
rf.fit(X_train[feature_cols], y_train)

# 6. Score validation candidates and select configuration using Recall@10 / NDCG@10
val_pairs['score'] = rf.predict_proba(X_val[feature_cols])[:,1]

# 7. Refit selected configuration on allowed development history, then evaluate once
# on the locked test window with the SAME candidate policy.

# 8. Save
joblib.dump(rf, 'models/random_forest.joblib')
```

Appendix B - Ranking Evaluation Functions

```
def evaluate_user(recs, relevant, k=10):
    r = list(recs)[:k]
    rel = set(relevant)
    hits = [1 if x in rel else 0 for x in r]
    precision = sum(hits) / k
    recall = sum(hits) / len(rel) if rel else np.nan
    hitrate = float(sum(hits) > 0)
    return {'precision': precision, 'recall': recall, 'hitrate': hitrate}

def average_precision_at_k(recs, relevant, k=10):
    rel = set(relevant)
    if not rel: return np.nan
    score, hits = 0.0, 0
    for rank, item in enumerate(list(recs)[:k], start=1):
        if item in rel:
            hits += 1
            score += hits / rank
    return score / min(len(rel), k)
```

Appendix C - Acceptance Tests

```
assert train_hist.InvoiceDate.max() < train_end
assert val_future.InvoiceDate.min() >= train_end
assert test_future.InvoiceDate.min() >= val_end
assert set(test_future.index).isdisjoint(set(train_hist.index))
assert set(feature_cols).issubset(set(X_train.columns))
assert not X_train[feature_cols].isna().any().any()
assert set(np.unique(y_train)).issubset({0,1})
assert rf.classes_.tolist() == [0,1]
assert all(len(r) <= K for r in recommendation_lists)
assert 500 <= len(eligible_candidate_items) <= 2000  # core instructor-approved range; adjust only if documented

assert 0.0 <= candidate_recall <= 1.0

assert advanced_result_seeds >= 3 or bootstrap_ci_reported  # advanced comparison

assert saved_model predictions match in-memory model on a verification sample
```

Appendix D - Advanced Recommender Comparison Protocol

Requirement	Protocol
Same split	All systems use the same chronological train/validation/test windows.
Same target	Relevant future purchases/interactions defined identically.
Same candidate universe	Where technically possible, evaluate the same candidate set.
Metrics	Recall@K, NDCG@K, MAP@K, coverage, runtime/latency.
Multiple seeds	Use at least 3 seeds for stochastic neural/latent models or report bootstrap CI.
Efficiency	Report training time, model size and approximate serving cost.
Interpretation	Explain whether a more complex model produces a practically meaningful gain.
Required benchmark	At least one of item-item collaborative filtering or matrix factorization/ALS must be compared with Random Forest.
Primary metrics	Recall@K and NDCG@K; candidate recall is reported separately. ROC-AUC/PR-AUC remain scorer diagnostics.
Uncertainty	Use at least three random seeds for stochastic models or a user-level bootstrap 95% confidence interval for the principal ranking metric.
Efficiency	Report training time, scoring latency per user, model size, and catalog coverage alongside ranking quality.

Appendix E - Instructor Setup Notes

- Provide a frozen CSV/parquet extract with file hash/version to avoid live-download delays, API/login problems, and inconsistent student datasets.
- For UCI Online Retail, predefine exact chronological train/validation/test cutoffs, minimum customer-history threshold, and repeat-purchase eligibility policy.
- Limit the candidate catalog to approximately 500-2,000 eligible items for the three-hour core so scoring remains computationally feasible and comparable across groups.
- Precompute or cache a feature-building helper if class time is limited; students should still understand and inspect the features.
- Ensure every future positive can appear in the evaluation candidate universe whenever feasible; otherwise require explicit candidate-recall reporting and retain excluded positives in the audit.
- Use CPU-friendly Random Forest settings for the core; reserve matrix factorization/neural methods for extensions.
- Provide a rubric-aligned starter notebook but leave candidate policy, feature interpretation, error analysis and model comparison as student work.